Hardware Bypass: executable notebook

This notebook turns **"Hardware Bypass: CSA Decomposition and GF(2) Jacobian Rank Deficits in the Topological Inversion of SHA-256"** into a runnable lab.

What this notebook does

1. Rebuilds the SHA-256 single-block round engine from scratch.
2. Instruments every round: `T1`, `T2`, carries, working variables, and message schedule words.
3. Verifies concrete identities that *can* be tested directly:
   - standard digest agreement with `hashlib`
   - the round-0 ground witness `T2[0] = 0x08909ae5`
   - exact carry-split identity for modular addition
   - the differential coupling `a[i+1] - e[i+1] ≡ T2[i] - d[i] (mod 2^32)`
   - the cube-root origin of the SHA-256 `K` constants
4. Builds a clean scaffold for the GF(2) Jacobian / parity-filter work.

What this notebook does **not** fake

The paper asserts an **exact 188/192 GF(2) Jacobian rank deficit** and a corresponding four-invariant parity filter. The uploaded `.docx` describes that claim conceptually, but it does **not** pin down the reduced 192-bit operator explicitly enough to reproduce that number unambiguously from the paper text alone. So this notebook includes the **machinery** for rank / nullspace analysis and leaves the operator explicit and editable instead of pretending the 188/192 result has already been re-derived.

In [1]:
```
from __future__ import annotations

import csv
import hashlib
import json
import math
import random
```

```
import statistics
from decimal import Decimal, getcontext
from pathlib import Path
from typing import Callable, Iterable, List, Dict, Tuple

try:
    import pandas as pd
except Exception:
    pd = None

MASK32 = 0xFFFFFFFF

# ---------------------------------------------------------------
# CONFIG
# ---------------------------------------------------------------
SEED = 1337
RNG = random.Random(SEED)

SAMPLE_MESSAGES = 200
MAX_SINGLE_BLOCK_BYTES = 55  # keep all experiments in the one-block regime
OUTPUT_DIR = Path("cryptographic_unbraiding_outputs")
OUTPUT_DIR.mkdir(exist_ok=True)

getcontext().prec = 80
```

In [2]:

```
# ---------------------------------------------------------------
# SHA-256 constants
# ---------------------------------------------------------------
H0 = [
    0x6A09E667, 0xBB67AE85, 0x3C6EF372, 0xA54FF53A,
    0x510E527F, 0x9B05688C, 0x1F83D9AB, 0x5BE0CD19,
]

K = [
    0x428A2F98, 0x71374491, 0xB5C0FBCF, 0xE9B5DBA5,
    0x3956C25B, 0x59F111F1, 0x923F82A4, 0xAB1C5ED5,
    0xD807AA98, 0x12835B01, 0x243185BE, 0x550C7DC3,
    0x72BE5D74, 0x80DEB1FE, 0x9BDC06A7, 0xC19BF174,
    0xE49B69C1, 0xEFBE4786, 0x0FC19DC6, 0x240CA1CC,
    0x2DE92C6F, 0x4A7484AA, 0x5CB0A9DC, 0x76F988DA,
    0x983E5152, 0xA831C66D, 0xB00327C8, 0xBF597FC7,
    0xC6E00BF3, 0xD5A79147, 0x06CA6351, 0x14292967,
```

```
    0x27B70A85, 0x2E1B2138, 0x4D2C6DFC, 0x53380D13,
    0x650A7354, 0x766A0ABB, 0x81C2C92E, 0x92722C85,
    0xA2BFE8A1, 0xA81A664B, 0xC24B8B70, 0xC76C51A3,
    0xD192E819, 0xD6990624, 0xF40E3585, 0x106AA070,
    0x19A4C116, 0x1E376C08, 0x2748774C, 0x34B0BCB5,
    0x391C0CB3, 0x4ED8AA4A, 0x5B9CCA4F, 0x682E6FF3,
    0x748F82EE, 0x78A5636F, 0x84C87814, 0x8CC70208,
    0x90BEFFFA, 0xA4506CEB, 0xBEF9A3F7, 0xC67178F2,
]
```

In [3]:

```python
# ------------------------------------------------------------
# Core bit operations
# ------------------------------------------------------------
def rotr(x: int, n: int) -> int:
    return ((x >> n) | ((x << (32 - n)) & MASK32)) & MASK32

def shr(x: int, n: int) -> int:
    return (x >> n) & MASK32

def Sigma0(x: int) -> int:
    return rotr(x, 2) ^ rotr(x, 13) ^ rotr(x, 22)

def Sigma1(x: int) -> int:
    return rotr(x, 6) ^ rotr(x, 11) ^ rotr(x, 25)

def sigma0(x: int) -> int:
    return rotr(x, 7) ^ rotr(x, 18) ^ shr(x, 3)

def sigma1(x: int) -> int:
    return rotr(x, 17) ^ rotr(x, 19) ^ shr(x, 10)

def Ch(x: int, y: int, z: int) -> int:
    return (x & y) ^ ((~x & MASK32) & z)

def Maj(x: int, y: int, z: int) -> int:
    return (x & y) ^ (x & z) ^ (y & z)

def add32_raw(*vals: int) -> Tuple[int, int]:
    total = sum(vals)
    return total & MASK32, total >> 32

def add32(*vals: int) -> int:
```

```
    return add32_raw(*vals)[0]

def pad_single_block(msg: bytes) -> bytes:
    if len(msg) > MAX_SINGLE_BLOCK_BYTES:
        raise ValueError(f"message too long for one-block mode: {len(msg)} bytes")
    bit_len = len(msg) * 8
    data = msg + b"\x80"
    while len(data) % 64 != 56:
        data += b"\x00"
    data += bit_len.to_bytes(8, "big")
    assert len(data) == 64
    return data

def message_schedule(block: bytes) -> List[int]:
    W = [int.from_bytes(block[i*4:(i+1)*4], "big") for i in range(16)]
    for t in range(16, 64):
        W.append(add32(sigma1(W[t-2]), W[t-7], sigma0(W[t-15]), W[t-16]))
    return W
```

In [4]:

```
# ------------------------------------------------------------
# Instrumented one-block SHA-256 trace
# ------------------------------------------------------------
def sha256_trace_single_block(msg: bytes | None = None, W_override: List[int] | None = None) -> Tuple[List[Dict[str,
    if W_override is None:
        if msg is None:
            raise ValueError("Provide either msg or W_override")
        block = pad_single_block(msg)
        W = message_schedule(block)
    else:
        if len(W_override) != 64:
            raise ValueError("W_override must contain exactly 64 words")
        W = list(W_override)

    a, b, c, d, e, f, g, h = H0
    rounds: List[Dict[str, int]] = []

    for t in range(64):
        S1 = Sigma1(e)
        ch = Ch(e, f, g)
        T1, T1_carry = add32_raw(h, S1, ch, K[t], W[t])

        S0 = Sigma0(a)
```

```
        maj = Maj(a, b, c)
        T2, T2_carry = add32_raw(S0, maj)

        a_next = add32(T1, T2)
        e_next = add32(d, T1)

        rounds.append({
            "round": t,
            "a": a, "b": b, "c": c, "d": d,
            "e": e, "f": f, "g": g, "h": h,
            "W": W[t],
            "S0": S0, "S1": S1,
            "Ch": ch, "Maj": maj,
            "T1": T1, "T2": T2,
            "T1_carry": T1_carry,
            "T2_carry": T2_carry,
            "a_next": a_next,
            "e_next": e_next,
        })

        a, b, c, d, e, f, g, h = a_next, a, b, c, e_next, e, f, g

    digest_words = [add32(x, y) for x, y in zip(H0, [a, b, c, d, e, f, g, h])]
    digest_hex = b"".join(x.to_bytes(4, "big") for x in digest_words).hex()
    return rounds, digest_hex
```

In [5]:

```
# ------------------------------------------------------------
# Validation against hashlib
# ------------------------------------------------------------
test_messages = [
    b"",
    b"abc",
    b"hello",
    b"The quick brown fox jumps over the lazy dog",
    b"a" * 55,
]

for msg in test_messages:
    traced = sha256_trace_single_block(msg)[1]
    reference = hashlib.sha256(msg).hexdigest()
    assert traced == reference, (msg, traced, reference)
```

```
print("All one-block validation tests matched hashlib.")
```

```
All one-block validation tests matched hashlib.
```

In [6]:
```
# ------------------------------------------------------------
# A small visible round table for 'abc'
# ------------------------------------------------------------
abc_rounds, abc_digest = sha256_trace_single_block(b"abc")
print("digest('abc') =", abc_digest)

preview_rows = []
for r in abc_rounds[:4]:
    preview_rows.append({
        "round": r["round"],
        "W": f"0x{r['W']:08x}",
        "T1": f"0x{r['T1']:08x}",
        "T2": f"0x{r['T2']:08x}",
        "T1_carry": r["T1_carry"],
        "T2_carry": r["T2_carry"],
    })

if pd is not None:
    display(pd.DataFrame(preview_rows))
else:
    for row in preview_rows:
        print(row)
```

```
digest('abc') = ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad
```

	round	W	T1	T2	T1_carry	T2_carry
0	0	0x61626380	0x54da50e8	0x08909ae5	1	1
1	1	0x00000000	0x3c5f8617	0x1e0b5396	1	1
2	2	0x00000000	0x3dc18b66	0x8b01bc41	2	0
3	3	0x00000000	0xbad621e9	0x1a7ad47d	1	1

CSA decomposition

The exact two-input modular addition split is:

[x + y = (x \oplus y) + 2(x \wedge y)]

as an integer identity. Modulo (2^{32}), the same split gives a **linear-looking XOR stream** plus a **shifted carry stream**.

That is the simplest executable version of the paper's “hardware unbraiding” move.

In [7]:
```python
# ------------------------------------------------------------
# CSA / exact two-input split
# ------------------------------------------------------------
def csa_split_two(x: int, y: int) -> Tuple[int, int]:
    sum_stream = x ^ y
    carry_stream = ((x & y) << 1) & MASK32
    return sum_stream, carry_stream

# verify the identity on random pairs
for _ in range(10_000):
    x = RNG.getrandbits(32)
    y = RNG.getrandbits(32)
    s, c = csa_split_two(x, y)
    assert add32(x, y) == add32(s, c)

print("CSA two-input split verified on 10,000 random pairs.")

# demonstrate on round-0 T2 ingredients
a0, b0, c0 = H0[0], H0[1], H0[2]
s0 = Sigma0(a0)
m0 = Maj(a0, b0, c0)
sum_stream, carry_stream = csa_split_two(s0, m0)

print(f"Sigma0(H0[a]) = 0x{s0:08x}")
print(f"Maj(H0[a],H0[b],H0[c]) = 0x{m0:08x}")
print(f"sum_stream = 0x{sum_stream:08x}")
print(f"carry_stream = 0x{carry_stream:08x}")
print(f"recombined = 0x{add32(sum_stream, carry_stream):08x}")
```

```
CSA two-input split verified on 10,000 random pairs.
Sigma0(H0[a]) = 0xce20b47e
Maj(H0[a],H0[b],H0[c]) = 0x3a6fe667
sum_stream = 0xf44f5219
carry_stream = 0x144148cc
recombined = 0x08909ae5
```

Ground witness and NOP backbone

The uploaded paper and related SHA die notes emphasize the input-invariant round-0 witness:

[T2[0] = \Sigma_0(H_0[a]) + \mathrm{Maj}(H_0[a], H_0[b], H_0[c]) = 0x08909ae5]

We can verify that directly, then run a **NOP backbone** with `W[t] = 0` for all 64 rounds and inspect the T2 carry channel.

In [8]:
```python
# ------------------------------------------------------------
# Ground witness and NOP backbone
# ------------------------------------------------------------
ground_witness = abc_rounds[0]["T2"]
assert ground_witness == 0x08909AE5
print(f"T2[0] ground witness = 0x{ground_witness:08x}")

nop_rounds, _ = sha256_trace_single_block(W_override=[0] * 64)

t2_bits = [r["T2_carry"] & 1 for r in nop_rounds]
t2_bitstring = "".join(str(bit) for bit in t2_bits)

# two ways of packing the 64 carry bits:
# - msb_first matches the visible left-to-right bitstring
# - lsb_first matches the session note's hex packing
t2_signature_msb_first = int(t2_bitstring, 2)
t2_signature_lsb_first = sum(bit << i for i, bit in enumerate(t2_bits))

print("NOP T2 carry bitstring:")
print(t2_bitstring)
print()
print(f"NOP T2 carry signature (MSB-first pack) = 0x{t2_signature_msb_first:016x}")
print(f"NOP T2 carry signature (LSB-first pack) = 0x{t2_signature_lsb_first:016x}")
```

```
T2[0] ground witness = 0x08909ae5
NOP T2 carry bitstring:
11011110000110100101010001010110100011010110000011111011110110110

NOP T2 carry signature (MSB-first pack) = 0xde1a54568d60f7b6
NOP T2 carry signature (LSB-first pack) = 0x6def06b16a2a587b
```

Differential coupling identity

A concrete identity from the SHA die work is:

[a[i+1] - e[i+1] \equiv T2[i] - d[i] \pmod{2^{32}}]

This is algebraically exact:

- (a[i+1] = T1[i] + T2[i])
- (e[i+1] = d[i] + T1[i])

Subtract the two, and (T1[i]) cancels.

In [9]:
```python
# ------------------------------------------------------------
# Sziklai-style differential coupling test
# ------------------------------------------------------------
def deterministic_random_messages(count: int, max_len: int = MAX_SINGLE_BLOCK_BYTES, seed: int = SEED) -> List[bytes]
    rng = random.Random(seed)
    out = []
    for _ in range(count):
        n = rng.randint(0, max_len)
        out.append(bytes(rng.getrandbits(8) for _ in range(n)))
    return out

violations = 0
total_rounds = 0
for msg in deterministic_random_messages(500):
    rounds, _ = sha256_trace_single_block(msg)
    for r in rounds:
        lhs = (r["a_next"] - r["e_next"]) & MASK32
        rhs = (r["T2"] - r["d"]) & MASK32
        total_rounds += 1
```

```
        if lhs != rhs:
            violations += 1

print(f"Rounds checked: {total_rounds}")
print(f"Violations: {violations}")
assert violations == 0
```

```
Rounds checked: 32000
Violations: 0
```

Carry-channel sensitivity

This measures how far each message's 64-bit **T2 carry signature** is from the NOP backbone signature, using Hamming distance.

This does **not** prove inversion. It simply gives a clean empirical handle on how sensitive the carry channel is to input geometry.

In [10]:
```
# ------------------------------------------------------------
# Carry signature sensitivity
# ------------------------------------------------------------
def t2_carry_signature_lsb_first(rounds: List[Dict[str, int]]) -> int:
    return sum((r["T2_carry"] & 1) << i for i, r in enumerate(rounds))

def hamming_distance_64(x: int, y: int) -> int:
    return (x ^ y).bit_count()

baseline = t2_signature_lsb_first
distances = []

for msg in deterministic_random_messages(SAMPLE_MESSAGES, seed=SEED + 1):
    rounds, _ = sha256_trace_single_block(msg)
    sig = t2_carry_signature_lsb_first(rounds)
    distances.append(hamming_distance_64(sig, baseline))

summary = {
    "sample_size": SAMPLE_MESSAGES,
    "mean_hamming_distance": statistics.mean(distances),
    "population_stddev": statistics.pstdev(distances),
    "min_distance": min(distances),
    "max_distance": max(distances),
}
```

```
print(json.dumps(summary, indent=2))
```

```
{
  "sample_size": 200,
  "mean_hamming_distance": 31.37,
  "population_stddev": 3.6692097241776738,
  "min_distance": 21,
  "max_distance": 42
}
```

Auditing the `K` constants

SHA-256's round constants are the first 32 bits of the fractional parts of the **cube roots of the first 64 primes.**

That is a hard, testable claim, and the notebook can verify it directly.

In [11]:
```
# ------------------------------------------------------------
# K-constant audit
# ------------------------------------------------------------
def first_n_primes(n: int) -> List[int]:
    primes = []
    candidate = 2
    while len(primes) < n:
        is_prime = True
        limit = int(candidate ** 0.5)
        for p in primes:
            if p > limit:
                break
            if candidate % p == 0:
                is_prime = False
                break
        if is_prime:
            primes.append(candidate)
        candidate += 1
    return primes

def cube_root_fraction_word(p: int) -> int:
    root = Decimal(p) ** (Decimal(1) / Decimal(3))
    frac = root - int(root)
```

```
    return int(frac * (1 << 32))

primes = first_n_primes(64)
k_audit_rows = []
for i, p in enumerate(primes):
    expected = cube_root_fraction_word(p)
    actual = K[i]
    k_audit_rows.append({
        "index": i,
        "prime": p,
        "expected": f"0x{expected:08x}",
        "actual": f"0x{actual:08x}",
        "match": expected == actual,
    })

assert all(row["match"] for row in k_audit_rows)
print("All 64 K constants match the cube-root prime construction.")

if pd is not None:
    display(pd.DataFrame(k_audit_rows[:8]))
else:
    for row in k_audit_rows[:8]:
        print(row)
```

```
All 64 K constants match the cube-root prime construction.
```

	index	prime	expected	actual	match
0	0	2	0x428a2f98	0x428a2f98	True
1	1	3	0x71374491	0x71374491	True
2	2	5	0xb5c0fbcf	0xb5c0fbcf	True
3	3	7	0xe9b5dba5	0xe9b5dba5	True
4	4	11	0x3956c25b	0x3956c25b	True
5	5	13	0x59f111f1	0x59f111f1	True
6	6	17	0x923f82a4	0x923f82a4	True
7	7	19	0xab1c5ed5	0xab1c5ed5	True

Why even Jacobian entries are a wall over (\mathbb{Z}/2^n\mathbb{Z})

The paper's 2-adic point is simple at the core:

- modulo (2^n), **odd** numbers have inverses
- **even** numbers do not

So any Newton / Hensel style step that needs division by an even derivative is dead on arrival in that ring.

This next cell gives a toy Hensel-lift demo that is completely standard.

In [12]:
```python
# ------------------------------------------------------------
# 2-adic / Hensel toy demonstration
# ------------------------------------------------------------
def inv_mod_power_of_two(a: int, nbits: int) -> int:
    modulus = 1 << nbits
    if a % 2 == 0:
        raise ValueError("even numbers are not invertible modulo 2^n")
    # Python's built-in modular inverse works here
    return pow(a, -1, modulus)

def hensel_step_linearized(fx: int, dfx: int, xk: int, nbits: int) -> int:
    modulus = 1 << nbits
    inv = inv_mod_power_of_two(dfx, nbits)
    return (xk - fx * inv) % modulus

# good case: odd derivative
xk = 1
fx = 3
dfx = 5
x_next = hensel_step_linearized(fx, dfx, xk, 8)
print(f"odd-derivative step succeeded: x_next = {x_next}")

# blocked case: even derivative
try:
    hensel_step_linearized(fx=3, dfx=6, xk=1, nbits=8)
except ValueError as exc:
```

```
    print("even-derivative step blocked exactly as expected:")
    print(" ", exc)
```

```
odd-derivative step succeeded: x_next = 154
even-derivative step blocked exactly as expected:
  even numbers are not invertible modulo 2^n
```

GF(2) Jacobian lab scaffold

This is the machinery needed for the rank-deficit / parity-filter part:

1. represent a word-vector operator
2. perturb each input bit
3. collect the output-difference columns
4. compute rank and nullity over GF(2)

Important: the paper claims an exact **188/192** result for a specific reduced operator, but the uploaded paper text does not define that operator explicitly enough to reconstruct it uniquely here. So the notebook provides a default **editable surrogate operator** plus the full rank/nullspace machinery.

In [13]:
```
# ------------------------------------------------------------
# GF(2) Jacobian utilities
# ------------------------------------------------------------
def words_to_int(words: List[int]) -> int:
    x = 0
    for w in words:
        x = (x << 32) | (w & MASK32)
    return x

def int_to_words(x: int, n_words: int) -> List[int]:
    return [(x >> (32 * (n_words - 1 - i))) & MASK32 for i in range(n_words)]

def gf2_rank_from_columns(columns: List[int], n_rows: int) -> int:
    basis = [0] * n_rows
    rank = 0
    for col in columns:
        x = col
        while x:
            pivot = x.bit_length() - 1
```

```
            if basis[pivot]:
                x ^= basis[pivot]
            else:
                basis[pivot] = x
                rank += 1
                break
    return rank

def jacobian_columns_gf2(
    func: Callable[[List[int]], List[int]],
    x_words: List[int],
) -> Tuple[List[int], int]:
    n_in_bits = len(x_words) * 32
    base_out = words_to_int(func(x_words))
    x0 = words_to_int(x_words)
    columns: List[int] = []
    for j in range(n_in_bits):
        xj = x0 ^ (1 << (n_in_bits - 1 - j))
        yj = words_to_int(func(int_to_words(xj, len(x_words))))
        columns.append(base_out ^ yj)
    return columns, n_in_bits
```

In [14]:

```
# ------------------------------------------------------------
# Two GF(2) operator demonstrations
# ------------------------------------------------------------
# (A) Architectural surrogate:
#     a carry-free baseline that preserves the paper's dual-path intuition.
#
# (B) Toy rank-188 operator:
#     a deliberately constructed 192-bit map with four collapsed directions.
#     This is NOT the paper's proof. It simply demonstrates that the rank /
#     nullity machinery behaves exactly as expected when the operator itself
#     really does have a 4-dimensional deficit.

def xor_channel_surrogate(words: List[int]) -> List[int]:
    a, b, c, d, e, h = words
    t1_lin = h ^ Sigma1(e)
    t2_lin = Sigma0(a) ^ b ^ c

    out0 = t1_lin ^ t2_lin
    out1 = a
    out2 = b
```

```
    out3 = c
    out4 = d ^ t1_lin
    out5 = e
    return [out0, out1, out2, out3, out4, out5]

def toy_rank188_operator(words: List[int]) -> List[int]:
    x = words_to_int(words)

    # Project away exactly four independent directions:
    # the lowest four output bits are forced to zero.
    # That produces an operator of rank 188 over a 192-bit domain.
    y = x & ~0b1111

    return int_to_words(y, 6)

base_state_192 = [H0[0], H0[1], H0[2], H0[3], H0[4], H0[7]]

# (A) architectural surrogate
columns_surrogate, nbits = jacobian_columns_gf2(xor_channel_surrogate, base_state_192)
rank_surrogate = gf2_rank_from_columns(columns_surrogate, nbits)
nullity_surrogate = nbits - rank_surrogate

# (B) toy rank-188 demonstration
columns_toy, _ = jacobian_columns_gf2(toy_rank188_operator, base_state_192)
rank_toy = gf2_rank_from_columns(columns_toy, nbits)
nullity_toy = nbits - rank_toy

print(f"Architectural surrogate rank   = {rank_surrogate}")
print(f"Architectural surrogate nullity = {nullity_surrogate}")
print()
print(f"Toy rank-188 operator rank     = {rank_toy}")
print(f"Toy rank-188 operator nullity   = {nullity_toy}")
assert rank_toy == 188 and nullity_toy == 4
```

```
Architectural surrogate rank   = 192
Architectural surrogate nullity = 0

Toy rank-188 operator rank     = 188
Toy rank-188 operator nullity   = 4
```

The two outputs serve different purposes:

- **Architectural surrogate**: a clean carry-free baseline that keeps the paper's structure visible, but is **not** claimed to be the paper's exact reduced operator.
- **Toy rank-188 operator**: a controlled demonstration that the GF(2) rank/nullity machinery correctly detects a **4-dimensional deficit** when the operator actually has one.

So the notebook now has both:

1. a practical SHA-facing scaffold, and
2. a sanity-check proving the algebraic tooling is ready for the exact `188/192` target once that reduced map is nailed down.

In [15]:
```
# ------------------------------------------------------------
# Export useful artifacts
# ------------------------------------------------------------
# Round trace for 'abc'
with (OUTPUT_DIR / "abc_round_trace.csv").open("w", newline="") as f:
    writer = csv.DictWriter(f, fieldnames=list(abc_rounds[0].keys()))
    writer.writeheader()
    writer.writerows(abc_rounds)

# NOP backbone summary
nop_summary = {
    "ground_witness_T2_round0_hex": f"0x{ground_witness:08x}",
    "nop_t2_carry_bitstring": t2_bitstring,
    "nop_t2_signature_msb_first_hex": f"0x{t2_signature_msb_first:016x}",
    "nop_t2_signature_lsb_first_hex": f"0x{t2_signature_lsb_first:016x}",
}

with (OUTPUT_DIR / "nop_backbone_summary.json").open("w") as f:
    json.dump(nop_summary, f, indent=2)

with (OUTPUT_DIR / "carry_sensitivity_summary.json").open("w") as f:
    json.dump(summary, f, indent=2)

with (OUTPUT_DIR / "k_constant_audit_first8.json").open("w") as f:
    json.dump(k_audit_rows[:8], f, indent=2)

print("Wrote:")
for path in sorted(OUTPUT_DIR.iterdir()):
    print(" -", path)
```

```
Wrote:
 - cryptographic_unbraiding_outputs\abc_round_trace.csv
 - cryptographic_unbraiding_outputs\carry_sensitivity_summary.json
 - cryptographic_unbraiding_outputs\k_constant_audit_first8.json
 - cryptographic_unbraiding_outputs\nop_backbone_summary.json
```

Verified outputs from this notebook

- Standard one-block SHA-256 execution matches `hashlib`.
- `T2[0] = 0x08909ae5` is recovered exactly.
- The two-input CSA split is exact.
- The differential coupling `a[i+1] - e[i+1] ≡ T2[i] - d[i] (mod 2^32)` holds on the sampled runs.
- The `K` constants match the cube-root prime definition exactly.
- The carry-channel experiments are now reproducible instead of narrative-only.

Open hook left explicit

- The paper's exact **188/192 GF(2) Jacobian** remains an operator-definition problem, not a coding problem.
- The notebook already contains the rank/nullspace engine needed for it.
- Once the reduced map is pinned down, the parity-filter machinery can be dropped in directly.